

1 Daten vom Industrie Controller mit der VR Brille synchronisieren

1.1 OPC UA Client test in Standalone APK

Description:

In einem Unity projekt (Zu finden im Sandbox Folder) haben wir einen auf PC funktionierenden Build erstellt, mit dem unser Programm mit dem Industrie Controller kommunizieren kann. Das builden einer Standalone apk hat funktioniert, die Kommunikation zwischen Industrie Controller und VR Brille allerdings nicht

1.2 OPC UA Communication Failure in APK Build

Description:

Da OPC UA auf Standalone build nicht funktioniert, haben wir das script umgebaut, sodass Kommunikation jetzt über TCP funktioniert.

1.3 TCP Read-Funktion (Statusabfrage)

Description: Die Kommunikation über TCP ermöglicht das Auslesen (Read) des aktuellen Zustands der Lampen vom Industrie-Controller.

Nach erfolgreicher Verbindung zum Server wird kontinuierlich auf eingehende Nachrichten gewartet. Diese werden zeilenweise empfangen und in eine Queue geschrieben, um sie im Hauptthread von Unity weiterzuverarbeiten. Wenn ein Read Gesendet Wird: Server antwortet mit Zustandswerten der Lampen im folgenden Format:

```
room1/Lampe=True  
room2/Lampe=False  
room3/Lampe=True
```

1.4 TCP Write-Funktion (Befehle senden)

Description: Die Write-Funktion ermöglicht das Senden von Steuerbefehlen über TCP an den Industrie-Controller, um den Zustand einzelner Lampen zu setzen. Befehle werden als formatierte Strings über den bestehenden TCP-Stream gesendet.

```
::room1:SwitchValueGL=True  
::room2:SwitchValueGL=False  
::room3:SwitchValueGL=True
```

1.5 Lampen-Synchronisation zwischen Unity und Industrie-Controller

Description: Description: Die Synchronisation stellt sicher, dass der Lampenzustand in Unity jederzeit mit dem tatsächlichen Zustand am Industrie-Controller übereinstimmt. Dazu wird zyklisch ein Read-Befehl gesendet, die Antwort geparkt und der interne Unity-Zustand entsprechend aktualisiert. Ändert der Nutzer in Unity den Zustand einer Lampe, wird ein Write-Befehl gesendet und anschließend der Zustand per Read verifiziert.

```
Unity sendet READ
↓
Controller antwortet mit Zustandswerten
↓
Unity parst Antwort → aktualisiert lokalen Lampenzustand
↓
Nutzer ändert Lampenzustand in Unity
↓
Unity sendet WRITE (::room1:SwitchValueGL=True)
↓
Unity sendet erneut READ zur Verifikation
↓
Lokaler Zustand wird aktualisiert
```